PYTHON PROGRAMMING LANGUAGE

Python Became the Best Programming Language & fastest programming language. Python is used in Machine Learning, Data Science, Big Data, Web Development, Scripting, llm , generati ai everywhere we will learn pyton from start to end || basic to expert. if you are not done programm then that is totally fine. I will explain from starting from scratch. python software - pycharm || vs code || jupyter || spyder

PYTHON INTERPRETTER

IDE (INTEGRATED DEVELOPMENT ENVIRONMENT)

IDE (INTEGRATED DEVELOPMENT ENVIRONMENT) =>

- using IDE - one can write code, run the code, debug the code
- IDE takes care of interpreting the Python code, running python scripts, building executables, and debugging the applications.
- An IDE enables programmers to combine the different aspects of writing a computer program.
- if you wnated to be python developer only then you need to install (IDE -- PYCHARM)

PYTHON INTERPRETER --> What is Python interpreter? A python interpreter is a computer program that converts each high-level program statement into machine code. An interpreter translates the command that you write out into code that the computer can understand

PYTHON INTERPRETER EXAMPLE --> You write your Python code in a text file with a name like hello.py . How does that code Run? There is program installed on your computer named "python3" or "python", and its job is looking at and running your Python code. This type of program is called an "interpreter".

PYTHON INTERPRETER & COMPILER

Both compilers and interpreters are used to convert a program written in a high-level language into machine code understood by computers. Interpreter -->

- Translates program one statement at a time
- Interpreter run every line item
- Execut the single, partial line of code
- Easy for programming

Compiler -->

- Scans the entire program and translates it as a whole into machine code.
- No execution if an error occurs
- you can not fix the bug (debug) line by line

Is Python an interpreter or compiler?

Python is an interpreted language, which means the source code of a Python program is converted into bytecode that is then executed by the Python virtual machine. Python is different from major compiled languages, such as C and C + +, as Python code is not required to be built and linked like code for these languages.

How to create python environment variable 1- cmd - python (if it not works) 2- find the location where the python is installed -- > C:\Users\A3MAX SOFTWARE TECH\AppData\Local\Programs\Python\Python39\Scripts 3- system -- env - environment variable screen will pop up 4- select on system variable - click on path - create New 5- C:\Users\kdata\AppData\Local\Programs\Python\Python311 6- env - sys variable - path - new - C:\Users\kdata\AppData\Local\Programs\Python\Python311\Scripts 7- cmd - type python -version 8- successfully python install in cmd

ANACONDA

Anaconda is a distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.), that aims to simplify package management and deployment.

In [1]:
```
1 + 1 # ADDITION
```

Out[1]: 2

In [2]:
```
2-1
```

Out[2]: 1

In [3]:
```
3*4
```

Out[3]: 12

In [4]:
```
8 / 4 # Division
```

Out[4]: 2.0

In [5]:
```
8 / 5 #float division
```

Out[5]: 1.6

In [6]:
```
8/4 ## float division
```

Out[6]: 2.0

In [7]:
```
8 // 4 #integer division
```

Out[7]: 2

In [8]:
```
8 + 9 - 7 # DMAS ( DIVISON, MUL, ADD, SUB)
```

Out[8]: 10

In [9]:
```
8 + 8 -  #syntax error:
```

```
  Cell In[9], line 1
    8 + 8 -  #syntax error:
             ^
SyntaxError: invalid syntax
```

In [10]:
```
5 + 5 * 5
```

Out[10]: 30

In [11]:
```
(5 + 5) * 5 # BODMAS (Bracket || Oders || Divide || Multiply || Add || Substact)
```

Out[11]: 50

In [12]:
```
2 * 2 * 2 * 2 * 2 # exponentaion
```

Out[12]: 32

In [13]:
```
2 * 5
```

Out[13]: 10

In [14]:
```
2 ** 5
```

Out[14]: 32

In [15]:
```
15 / 3
```

Out[15]: 5.0

In [16]:
```
10 // 3
```

Out[16]: 3

In [17]:
```
15 % 2 # Modulus
```

Out[17]: 1

In [18]:
```
10 % 2
```

Out[18]: 0

In [19]:
```
15 %% 2
```

```
  Cell In[19], line 1
    15 %% 2
       ^
SyntaxError: invalid syntax
```

In [20]:
```
3 + 'nit'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[20], line 1
----> 1 3 + 'nit'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [21]:
```
3 * 'nit'
```

Out[21]:
```
'nitnitnit'
```

In [22]:
```
3 * ' nit '
```

Out[22]:
```
' nit  nit  nit '
```

In [23]:
```
a,b,c,d,e = 15, 7.8, 'nit', 8+9j, True

print(a)
print(b)
print(c)
print(d)
print(e)
```

```
15
7.8
nit
(8+9j)
True
```

In [24]:
```
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'complex'>
<class 'bool'>
```

In [25]:
```
type(c)
```

Out[25]:
```
str
```

- So far we code with numbers(integer)

- Lets work with string

In [26]:
```
'Naresh IT'
```

Out[26]:
```
'Naresh IT'
```

python inbuild function - print & you need to pass the parameter in print()

A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

In [27]:
```
print('Max it')
```

```
Max it
```

In [28]:
```
"max it technology"
```

Out[28]:
```
'max it technology'
```

In [29]:
```
s1 = 'max it technology'
s1
```

Out[29]:
```
'max it technology'
```

In [30]:
```
a = 2
b = 3
a + b
```

Out[30]:
```
5
```

In [31]:
```
c = a + b
c
```

Out[31]:
```
5
```

In [32]:
```
a = 3
b = 'hi'
type(b)
```

Out[32]:
```
str
```

In [33]:
```
print('max it's"Technology"') # \ has some special meaning to ignore the error
```

```
  Cell In[33], line 1
    print('max it's"Technology"') # \ has some special meaning to ignore the error
                                ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [34]:
```
print('max it\'s"Technology"') #\ has some special meaning to ignore the error
```

```
max it's"Technology"
```

In [35]:
```
print('max it',  'Technology')
```

```
max it Technology
```

In [36]:
```
print("max it',  'Technology")
```

```
max it',  'Technology
```

In [37]:
```
# print the nit 2 times
'nit' + ' nit'
```

Out[37]: 'nit nit'

In [38]:
```
'nit'  ' nit'
```

Out[38]: 'nit nit'

In [39]:
```
#5 time print
5 * 'nit'
```

Out[39]: 'nitnitnitnitnit'

In [40]:
```
5*' nit' # soace between words
```

Out[40]: ' nit nit nit nit nit'

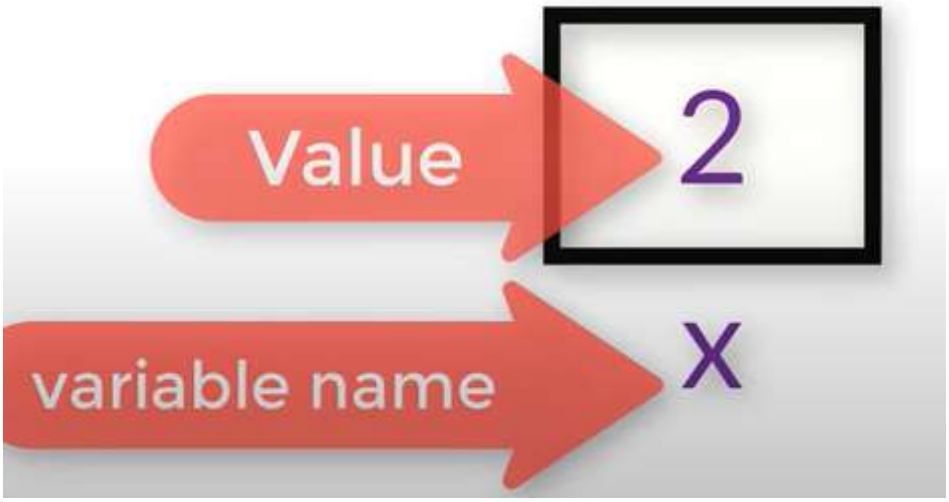
In [41]:
```
print('c:\nit') #\n -- new line # i will explain
```

```
c:
it
```

In [42]:
```
print(r'c:\nit') #raw string # i will explain later
```

```
c:\nit
```

variable || identifier || object



In [43]:
```
2
```

Out[43]: 2

In [44]:
```
x = 2 #x is variable/identifier/objec, 2 is the value
x
```

Out[44]: 2

In [45]:
```
x + 3
```

Out[45]: 5

In [46]:
```
y = 3
y
```

Out[46]: 3

In [47]:
```
x + y
```

Out[47]: 5

In [48]:
```
x = 9
x
```

Out[48]: 9

In [49]:
```
x + y
```

Out[49]: 12

In [50]:
```
x + 10
```

Out[50]: 19

In [51]:
```
_ + y # _ understand the previous result of the
```

Out[51]: 22

In [52]:
```
_ + y
```

Out[52]: 25

In [53]:
```
_ + y
```

Out[53]: 28

In [54]:
```
_ + y
```

Out[54]: 31

In [55]:
```
y
```

Out[55]: 3

In [56]:
```
_ + y
```

Out[56]: 6

In [57]:
```
_ + y
```

Out[57]: 9

In [58]:
```
_ + y
```

Out[58]: 12

In [59]:
```
# string variable
name = 'mit'
```

In [60]:
```
name
```

Out[60]: 'mit'

In [61]:
```
name + 'technology'
```

Out[61]: 'mittechnology'

In [62]:
```
name + ' technology'
```

Out[62]: 'mit technology'

In [63]:
```
name 'technology'
```

```
  Cell In[63], line 1
    name 'technology'
         ^
SyntaxError: invalid syntax
```

In [ ]:
```
name
```

In [64]:
```
len(name)
```

Out[64]: 3

In [65]:
```
name[0] #python index begins with 0
```

Out[65]: 'm'

In [66]:
```
name
```

Out[66]: 'mit'

In [67]:
```
name[5]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[67], line 1
----> 1 name[5]

IndexError: string index out of range
```

In [ ]:
```
name[7]
```

In [68]:
```
name[-1]
```

Out[68]:
```
't'
```

In [69]:
```
name[-2]
```

Out[69]:
```
'i'
```

In [70]:
```
name[-6]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[70], line 1
----> 1 name[-6]

IndexError: string index out of range
```

slicing

In [71]:
```
name
```

Out[71]:
```
'mit'
```

In [72]:
```
name[0:1] #to print 2 character
```

Out[72]:
```
'm'
```

In [73]:
```
name[0:2]
```

Out[73]:
```
'mi'
```

In [74]:
```
name[1:4]
```

Out[74]:
```
'it'
```

In [75]:
```
name[1:]
```

Out[75]:
```
'it'
```

In [76]:
```
name[:4]
```

Out[76]:
```
'mit'
```

In [77]:
```
name[3:9]
```

Out[77]:
```
''
```

In [78]:
```
name
```

Out[78]:
```
'mit'
```

In [79]:
```
name1 = 'fine' # change the string fine to dine
name1
```

Out[79]:
```
'fine'
```

In [80]:
```
name1[0:1]
```

Out[80]:
```
'f'
```

In [81]:
```
name1[0:1] = 'd' # i want to change 1st character of naresh (n) - t
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[81], line 1
----> 1 name1[0:1] = 'd' # i want to change 1st character of naresh (n) - t

TypeError: 'str' object does not support item assignment
```

In [ ]:
```
name1[0] = 'd' #strings in python are immutable
```

In [82]:
```
name1
```

Out[82]:
```
'fine'
```

In [83]:
```
name1[1:]
```

Out[83]:
```
'ine'
```

In [84]:
```
'd' + name1[1:] #i want to change fine to dine
```

Out[84]:
```
'dine'
```

In [85]:
```
len(name1) #python inbuild function
```

Out[85]:
```
4
```

List

In [86]:
```
l = []
```

In [87]:
```
# LIST LIST LIST
nums = [10,20,30]
nums
```

Out[87]: [10, 20, 30]

In [88]:
```
nums[0]
```

Out[88]: 10

In [89]:
```
nums[-1]
```

Out[89]: 30

In [90]:
```
nums[1:]
```

Out[90]: [20, 30]

In [91]:
```
nums[:1]
```

Out[91]: [10]

In [92]:
```
num1 = ['hi', 'hallo']
```

In [93]:
```
num1
```

Out[93]: ['hi', 'hallo']

In [94]:
```
num2 = ['hi', 8.9, 34] # we can assign multiple variable
num2
```

Out[94]: ['hi', 8.9, 34]

In [95]:
```
# can we have 2 list together
num3 = [nums, num1]
```

In [96]:
```
num3
```

Out[96]: [[10, 20, 30], ['hi', 'hallo']]

In [97]:
```
num4 = [nums, num1, num2]
```

In [98]:
```
num4
```

Out[98]: [[10, 20, 30], ['hi', 'hallo'], ['hi', 8.9, 34]]

In [99]:
```
nums
```

Out[99]: [10, 20, 30]

In [100…
```
nums.append(45)
```

In [101…
```
nums
```

Out[101… [10, 20, 30, 45]

In [102…
```
nums.remove(45)
```

In [103…
```
nums
```

Out[103… [10, 20, 30]

In [104…
```
nums.pop(1)
```

Out[104… 20

In [105…
```
nums
```

Out[105… [10, 30]

In [106…
```
nums.pop() #if you dont assign the index element then it will consider by default l
```

Out[106… 30

In [107…
```
nums
```

Out[107… [10]

In [108…
```
num1
```

Out[108… ['hi', 'hallo']

In [109…
```
num1.insert(2,'nit') #insert the value as per index values i.e 2nd index we are ass
```

In [110…
```
num1
```

Out[110… ['hi', 'hallo', 'nit']

In [111…
```
num1.insert(0, 1)
```

In [112…
```
num1
```

Out[112… [1, 'hi', 'hallo', 'nit']

In [113…
```
#if you want to delate multiple value
num2
```

Out[113… ['hi', 8.9, 34]

In [114…
```
del num2[2:]
```

In [115…
```
num2
```

Out[115… ['hi', 8.9]

In [116…
```
# if you need to add multiple values
num2.extend([29,15,20])
```

In [117…
```
num2
```
Out[117…
```
['hi', 8.9, 29, 15, 20]
```
In [118…
```
num3
```
Out[118…
```
[[10], [1, 'hi', 'hallo', 'nit']]
```
In [119…
```
num3.extend(['a', 5, 6.7])
```
In [120…
```
num3
```
Out[120…
```
[[10], [1, 'hi', 'hallo', 'nit'], 'a', 5, 6.7]
```
In [121…
```
nums
```
Out[121…
```
[10]
```
In [122…
```
min(nums) #inbuild function
```
Out[122…
```
10
```
In [123…
```
max(nums) #inbuild function
```
Out[123…
```
10
```
In [124…
```
num1
```
Out[124…
```
[1, 'hi', 'hallo', 'nit']
```
In [125…
```
min(num1)
```
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[125], line 1
----> 1 min(num1)

TypeError: '<' not supported between instances of 'str' and 'int'
```
In [126…
```
sum(nums) #inbuild function
```
Out[126…
```
10
```
In [127…
```
nums.sort() #sort method
```
In [128…
```
nums
```
Out[128…
```
[10]
```
In [129…
```
l = [1,2,3]
l
```

Out[129… [1, 2, 3]

In [130…
```
l[0] = 100
l
```

Out[130… [100, 2, 3]

Tuple

In [131…
```
# TUPLE TUPLE TUPLE
tup = (15,25,35)
tup
```

Out[131… (15, 25, 35)

In [132…
```
tup[0]
```

Out[132… 15

In [133…
```
tup[0] = 10
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[133], line 1
----> 1 tup[0] = 10

TypeError: 'tuple' object does not support item assignment
```

as we are unable to change any value or parameter in tuple so iteration very faster in tuple compare to list

SET

In [134…
```
# SET SET SET
S = {}
```

In [135…
```
s1 = {21,6,34,58,5}
```

In [136…
```
s1
```

Out[136… {5, 6, 21, 34, 58}

In [137…
```
s3= {50,35,53,'nit', 53}
```

In [138…
```
s3
```

Out[138… {35, 50, 53, 'nit'}

In [139…
```
s1[1] #as we dont have proper sequencing thats why indexing not subscriptable
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[139], line 1
----> 1 s1[1] #as we dont have proper sequencing thats why indexing not subscriptabl
e

TypeError: 'set' object is not subscriptable
```

DICTIONARY

In [140…
```
# DICTIONARY DICTIONARY DICTIONARY
data = {1:'apple', 2:'banana',4:'orange'}
data
```

Out[140… `{1: 'apple', 2: 'banana', 4: 'orange'}`

In [141…
```
data[4]
```

Out[141… `'orange'`

In [142…
```
data[3]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[142], line 1
----> 1 data[3]

KeyError: 3
```

In [143…
```
data.get(2)
```

Out[143… `'banana'`

In [144…
```
data.get(3)
```

In [145…
```
print(data.get(3))
```

```
None
```

In [146…
```
data.get(1,'Not Fount')
```

Out[146… `'apple'`

In [147…
```
data.get(3,'Not Found')
```

Out[147… `'Not Found'`

In [148…
```
data[5] = 'five'
```

In [149…
```
data
```

Out[149… `{1: 'apple', 2: 'banana', 4: 'orange', 5: 'five'}`

In [150…
```
del data [5]
```

In [151…
```
data
```

Out[151…
```
{1: 'apple', 2: 'banana', 4: 'orange'}
```

In [152…
```
#list in the dictionary
prog = {'python':['vscode', 'pycharm'], 'machine learning' : 'sklearn', 'datascienc
```

In [153…
```
prog
```

Out[153…
```
{'python': ['vscode', 'pycharm'],
 'machine learning': 'sklearn',
 'datascience': ['jupyter', 'spyder']}
```

In [154…
```
prog['python']
```

Out[154…
```
['vscode', 'pycharm']
```

In [155…
```
prog['machine learning']
```

Out[155…
```
'sklearn'
```

In [156…
```
prog['datascience']
```

Out[156…
```
['jupyter', 'spyder']
```

How to create environment variable

to find help

STEPS TO FIND HELP OPTION --> 1- help() | 2- topics | 3- search as per requirments | 4- quit
if you want help on any command then help(list) || help(tuple)

In [ ]:
```
help(list)
```

In [ ]:
```
2 + 3
```

In [ ]:
```
help(tuple)
```

introduce to ID()

In [157…
```
# variable address
num = 5
id(num)
```

Out[157…
```
140707842811128
```

In [158…
```
name = 'nit'
id(name) #Address will be different for both
```

Out[158…
```
1281548693936
```

In [159…
```
a = 10
id(a)
```

Out[159…
```
140707842811288
```

In [160…
```
b = a #thats why python is more memory efficient
```

In [161…
```
id(b)
```

Out[161…
```
140707842811288
```

In [162…
```
id(10)
```

Out[162…
```
140707842811288
```

In [163…
```
k = 10
id(k)
```

Out[163…
```
140707842811288
```

In [164…
```
a = 20  # as we change the value of a then address will change
id(a)
```

Out[164…
```
140707842811608
```

In [165…
```
id(b)
```

Out[165…
```
140707842811288
```

what ever the variale we assigned the memory and we not assigned anywhere then we can use as garbage collection.

-can we make VARIABLE as a CONSTANT (note - in python you cannot make variable as constant)

In [166…
```
PI = 3.14  #in math this is alway constant but python we can chang
PI
```

Out[166…
```
3.14
```

In [167…
```
PI = 3.15
PI
```

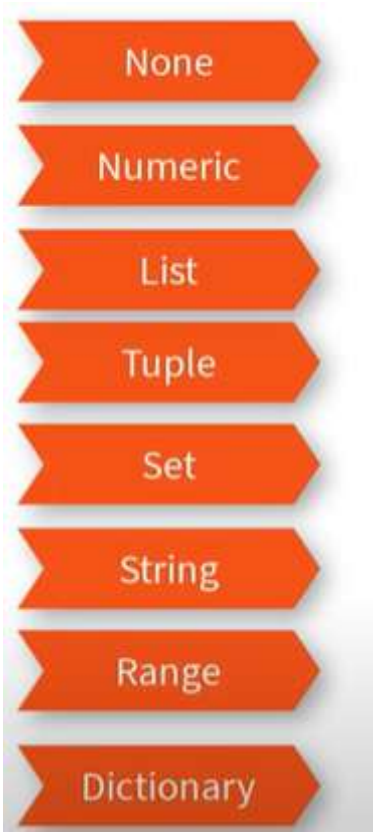
Out[167…
```
3.15
```

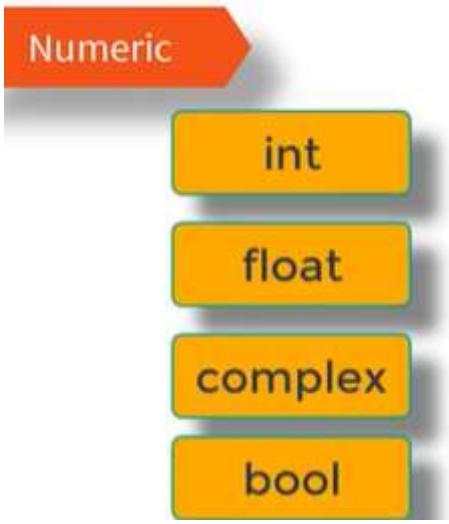
In [168…
```
type(PI)
```

Out[168… float

DATA TYPES & DATA STRUCTURES-->

1- NUMERIC || 2-LIST || 3-TUPLE || 4-SET || 5-STRING || 6-RANGE || 7-DICTIONARY





1- NUMERIC :- INT || FLOAT || COMPLEX || BOOL

In [169…
```
w = 2.5
type(w)
```

Out[169… float

In [170…
```
(a)
```

Out[170… 20

In [171…
```
w2 = 2 + 3j #so hear j is represent as root of -1
type(w2)
```

Out[171… complex

In [172…
```
#convert flot to integer
a = 5.6
b = int(a)
```

In [173…
```
b
```

Out[173… 5

In [174…
```
type(b)
```

Out[174… int

In [175…
```
type(a)
```

Out[175… float

In [176…
```
k = float(b)
```

In [177…
```
k
```

Out[177… 5.0

In [178…
```
print(a)
print(b)
print(k)
```

```
5.6
5
5.0
```

In [179…
```
k1 = complex(b,k)
```

In [180…
```
print(k1)
type(k1)
```

```
(5+5j)
```

Out[180… complex

In [181…
```
b < k
```

Out[181… False

In [182…
```
condition = b<k
condition
```

Out[182… False

In [183…
```
type(condition)
```

Out[183… bool

In [184…
```
int(True)
```

Out[184… 1

In [185…
```
int(False)
```

Out[185… 0

In [186…
```
l = [1,2,3,4]
print(l)
type(l)
```

```
[1, 2, 3, 4]
```

Out[186… list

In [187…
```
s = {1,2,3,4}
s
```

Out[187… {1, 2, 3, 4}

In [188…
```
type(s)
```

Out[188… set

In [189…
```
s1 = {1,2,3,4,4,3,11} #duplicates are not allowed
s1
```

Out[189… {1, 2, 3, 4, 11}

In [190…
```
t = (10,20,30)
t
```

Out[190… (10, 20, 30)

In [191…
```
type(t)
```

Out[191… tuple

In [192…
```
str = 'nit' #we dont have character in python
type(str)
```

Out[192… str

In [193…
```
st = 'n'
type(st)
```

Out[193… str

range()

In [194…
```
r = range(0,10)
r
```

Out[194… range(0, 10)

In [195…
```
type(r)
```

Out[195… range

In [196…
```
# if you want to print the range
list(range(0,10))
```

Out[196… [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

In [197…
```
r1 = list(r)
r1
```

Out[197… [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

In [198…
```
#if you want to print even number
even_number = list(range(2,10,2))
even_number
```

Out[198… [2, 4, 6, 8]

In [199…
```
d= {1:'one', 2:'two', 3:'three'}
d
```

Out[199… {1: 'one', 2: 'two', 3: 'three'}

In [200…
```
type(d)
```

Out[200… dict

In [201…
```
# print the keys
d.keys()
```

Out[201… dict_keys([1, 2, 3])

In [202…
```
d.values()
```

Out[202… dict_values(['one', 'two', 'three'])

In [203…
```
# how to get particular value
d[2]
```

Out[203… 'two'

In [204…
```
# other way to get value as
d.get(2)
```

Out[204…
```
'two'
```

operator

1- ARITHMETIC OPERATOR (+ , -, *, /, %, %%, **, ^ 2- ASSIGNMEN OPERATOR (=) 3- RELATIONAL OPERATOR 4- LOGICAL OPERATOR 5- UNARY OPERATOR



Arithmetic operator

In [205…
```
x1, y1 = 10, 5
```

In [206…
```
#x1 ^ y1
```

In [207…
```
x1  + y1
```

Out[207…
```
15
```

In [208…
```
x1 - y1
```

Out[208…
```
5
```

In [209…
```
x1 * y1
```

Out[209… 50

In [210…
```
x1 / y1
```

Out[210… 2.0

In [211…
```
x1 // y1
```

Out[211… 2

In [212…
```
x1 % y1
```

Out[212… 0

In [213…
```
x1 ** y1
```

Out[213… 100000

In [214…
```
x2 = 3
y2 = 3

x2 ** y2
```

Out[214… 27

Assignment operator

In [215…
```
x = 2
```

In [216…
```
x = x + 2 # if you want to increment by 2
```

In [217…
```
x
```

Out[217… 4

In [218…
```
x += 2
x
```

Out[218… 6

In [219…
```
x += 2
x
```

Out[219… 8

In [220…
```
x *= 2
```

In [221…
```
x
```

Out[221… 16

In [222…
```
x -= 2
```

In [223…
```
x
```

Out[223… 14

In [224…
```
x //= 2
x
```

Out[224… 7

In [225…
```
x /= 2
x
```

Out[225… 3.5

In [226…
```
a, b = 5,6 # you can assigned variable in one line as well
print(a)
print(b)
```

```
5
6
```

In [227…
```
a = 5
b = 6
print(a)
print(b)
```

```
5
6
```

In [228…
```
a
```

Out[228… 5

In [229…
```
b
```

Out[229… 6

unary operator

- unary means 1 || binary means 2
- Here we are applying unary minus operator(-) on the operand n; the value of m becomes -7, which indicates it as a negative value.

In [230…
```
n = 7 #negattion
n
```

Out[230… 7

In [231…
```
m = -(n)
```

```
m
```

Out[231… -7

In [232…
```
n
```

Out[232… 7

In [233…
```
-n
```

Out[233… -7

Relational operator

we are using this operator for comparing

In [234…
```
a = 5
b = 6
```

In [235…
```
a<b
```

Out[235… True

In [236…
```
a>b
```

Out[236… False

In [237…
```
# a = b # we cannot use = operatro that means it is assigning
```

In [238…
```
a == b
```

Out[238… False

In [239…
```
a1 = 4
b1 = 7
```

In [240…
```
a1 == b1
```

Out[240… False

In [241…
```
a1 != b1
```

Out[241… True

In [242…
```
a != b
```

Out[242… True

In [243…
```
# hear if i change b = 6
b = 5
```

In [244…
```
a == b
```

Out[244… True

In [245…
```
a
```

Out[245… 5

In [246…
```
b
```

Out[246… 5

In [247…
```
a >= b
```

Out[247… True

In [248…
```
a <= b
```

Out[248… True

In [249…
```
a < b
```

Out[249… False

In [250…
```
a>b
```

Out[250… False

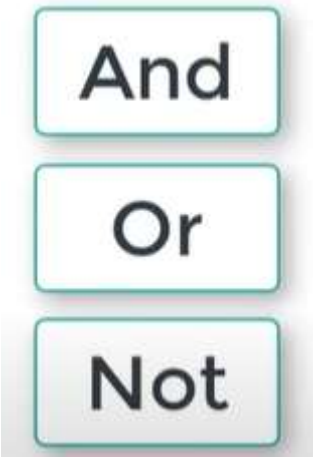
In [251…
```
b = 7
```

In [252…
```
a != b
```

Out[252… True

LOGICAL OPERATOR

- logical operator you need to understand about true & false table



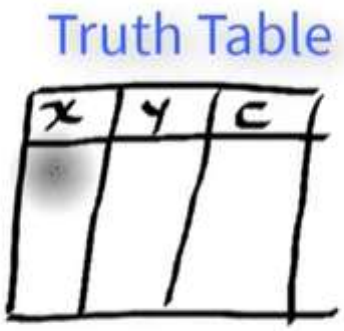
- 3 importand part of logical operator is --> AND, OR, NOT

- lets understand the truth table:- in truth table you can represent (true-1 & false means-0)

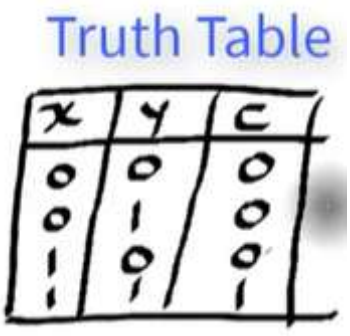
4

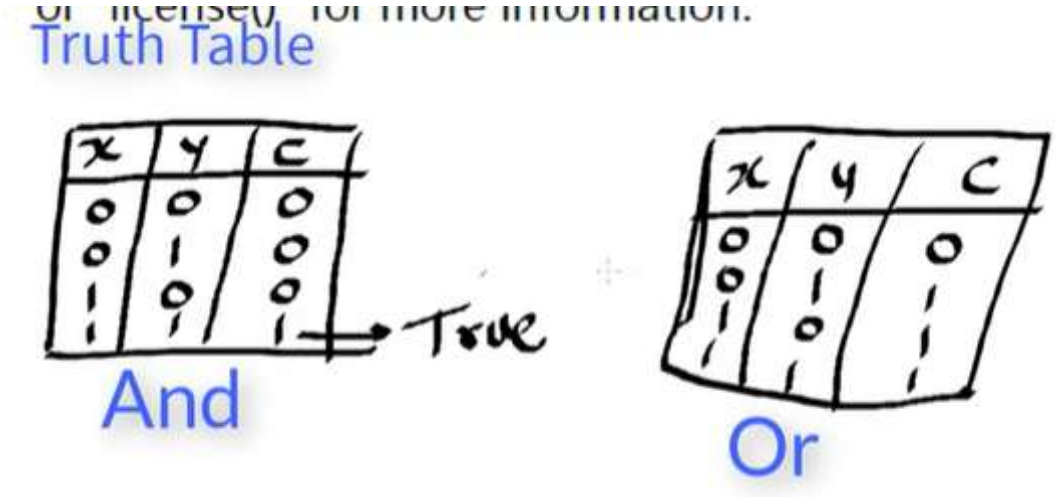
8 and b < 5

8 and b<2









In [253…
```
a = 5
b = 4
```

In [254…
```
a < 8 and b < 5 #refer to the truth table
```

Out[254… True

In [255…
```
a < 8 and b < 2
```

Out[255… False

In [256…
```
a
```

Out[256… 5

In [257…
```
b
```

Out[257… 4

In [258…
```
a < 8 or b < 2
```

Out[258… True

In [259…
```
a>8 or b<2
```

Out[259… False

In [260…
```
x = False
x
```

Out[260… False

In [261…
```
not x  # you can reverse the operation
```

Out[261… True

In [262…
```
x = not x
x
```

Out[262… True

In [263…
```
x
```

Out[263… True

In [264…
```
not x
```

Out[264… False

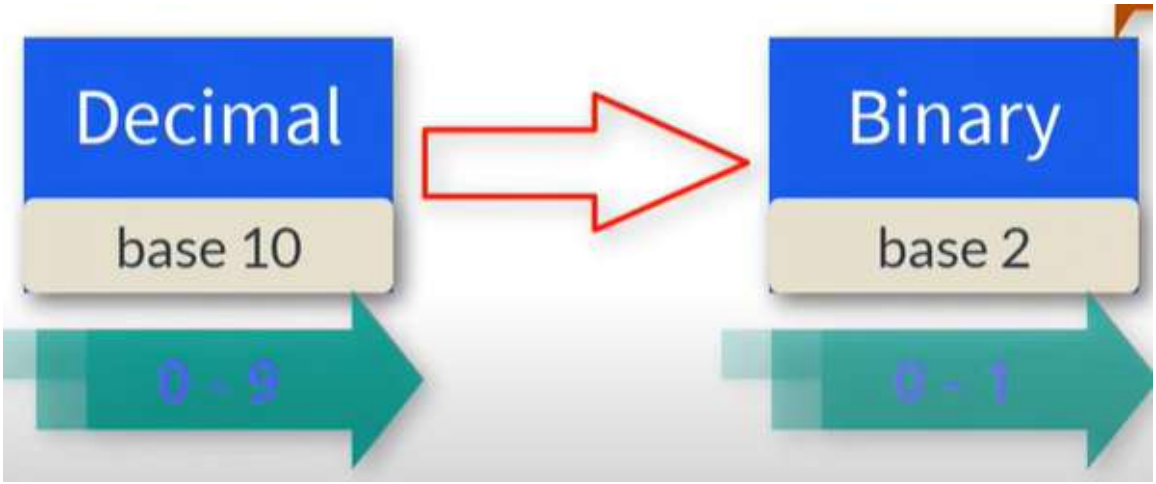
Number system coverstion (bit-binary digit)

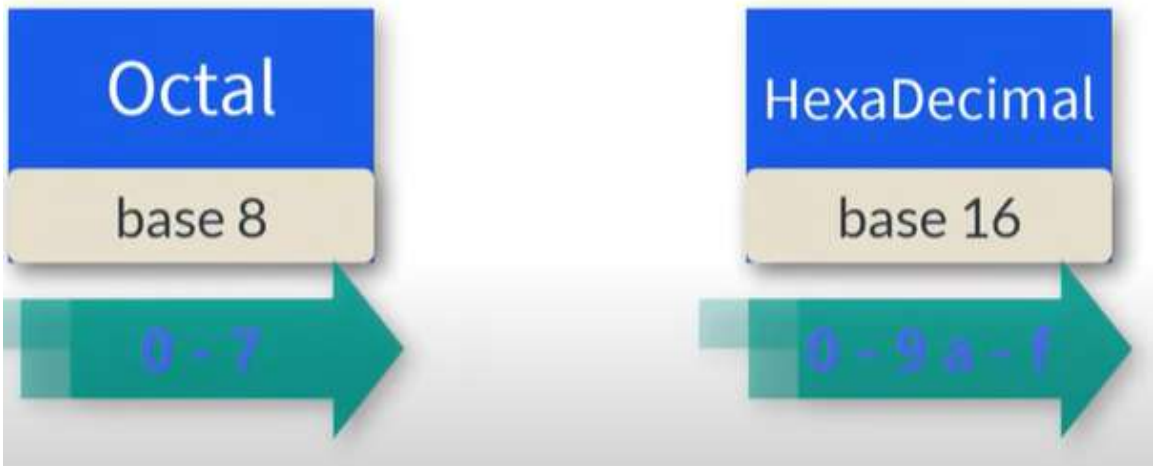
- In the programing we are using binary system, octal system, decimal system & hexadecimal system
- but where do we use this in cmd - you can check your ip address & lets understand how to convert from one system to other system
- when you check ipaddress you will these format --> cmd - ipconfig

```
Description . . . . . : Intel(R) Dual Band Wireless-AC
Physical Address. . . . . : 88-78-73-9E-74-38
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::4c59:48f6:38aa:660%3(Pref
```

binary : base (0-1) --> please divide 15/2 & count in reverse order ocatl : base (0-7)

hexadecimal : base (0-9 & then a-f) BIT -> BInary DigiT





In [265…
```
25
```
Out[265… 25

In [266…
```
bin(25)
```
Out[266… '0b11001'

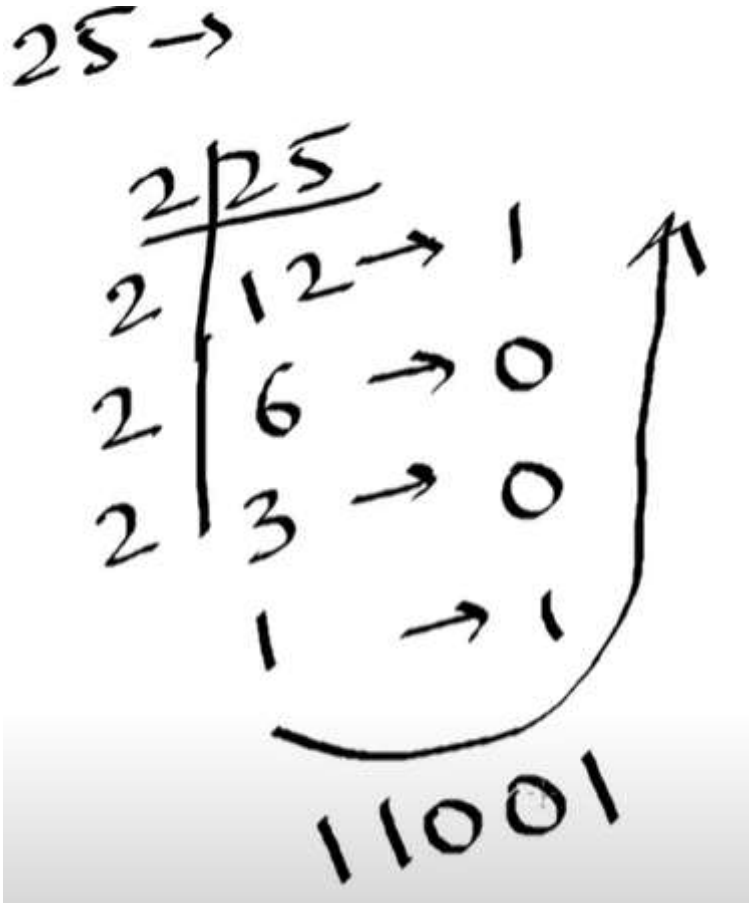
In [267…
```
int(0b11001)
```
Out[267… 25

In [268…
```
bin(30)
```
Out[268… '0b11110'

In [269…
```
int(0b11001)
```
Out[269… 25

In [270…
```
oct(25)
```
Out[270… '0o31'

In [271…
```
0o31
```
Out[271… 25



In [272…
```
0b11001
```
Out[272… 25

In [273…
```
int(0b11001)
```
Out[273… 25

In [274…
```
bin(7)
```
Out[274… '0b111'

In [275…
```
oct(25)
```
Out[275… '0o31'

In [276…
```
0o31
```
Out[276… 25

In [277…
```
int(0o31)
```
Out[277… 25

In [278…
```
hex(25)
```

Out[278… '0x19'

In [279…
```
0x19
```

Out[279… 25

In [280…
```
hex(16)
```

Out[280… '0x10'

In [281…
```
0xa
```

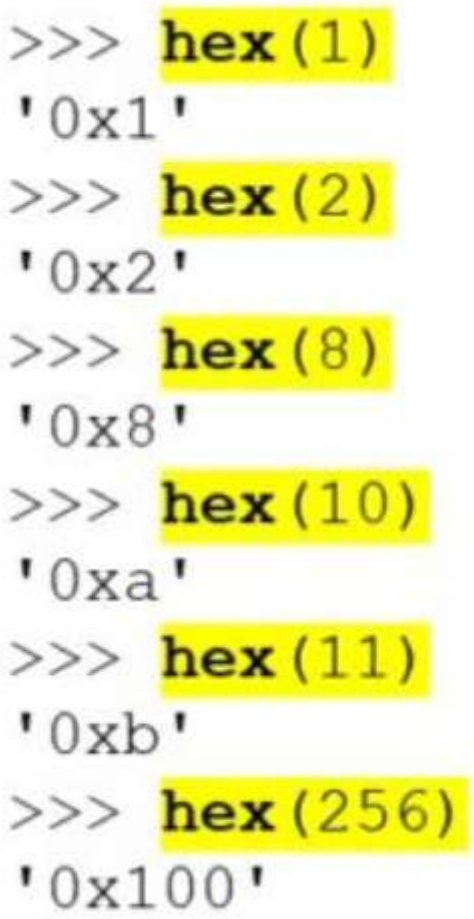
Out[281… 10

In [282…
```
0xb
```

Out[282… 11

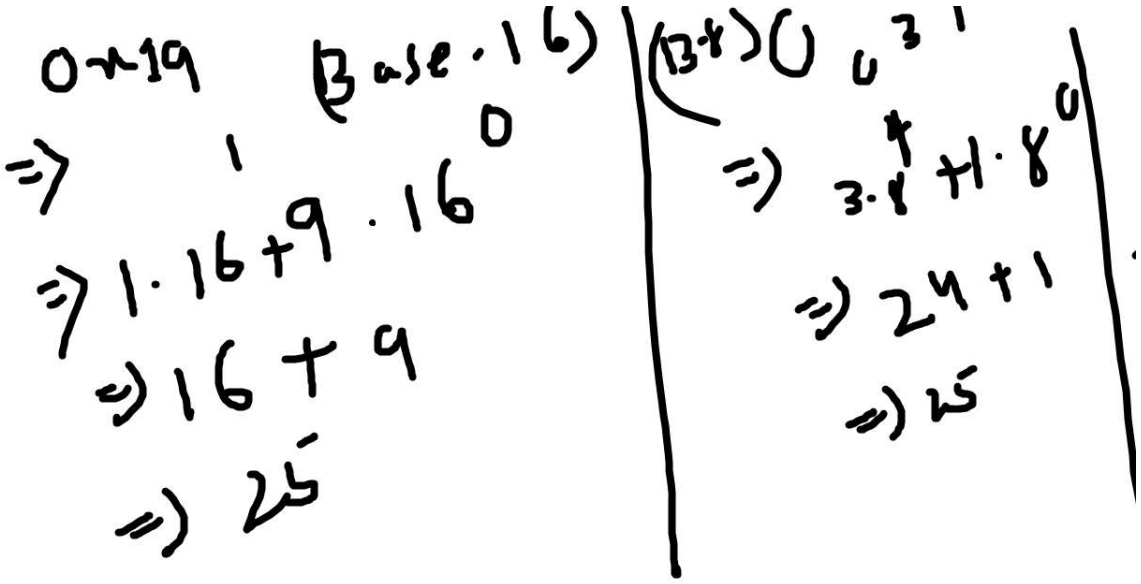
In [283…
```
hex(1)
```

Out[283… '0x1'



In [284…
```
hex(25)
```

Out[284… '0x19'



In [285… `int(0x19)`

Out[285… 25

In [286… `0x15`

Out[286… 21

In [287… `hex(25)`

Out[287… '0x19'

swap 2-variable in python

(a,b = 5,6) After swap we should get ==> (a, b = 6,5)

In [288…
```
a = 5
b = 6
```

In [289…
```
a = b
b = a
```

In [290…
```
print(a)
print(b)
```

```
6
6
```

In [291…
```
# in above scenario we lost the value 5
a1 = 7
b1 = 8
```

In [292…
```
temp = a1
a1 = b1
b1 = temp
```

In [293…
```
print(a1)
print(b1)
```

```
8
7
```

- in the above code we are using third variable
- in interview they might ask can we swap better way without using 3rd variable



In [294…
```
a2 = 5
b2 = 6
```

In [295…
```
#swap variable formulas without using 3rd formul
a2 = a2 + b2 # 5+6 = 11
b2 = a2 - b2 # 11-6 = 5
a2 = a2 - b2 # 11-5 = 6
```

In [296…
```
print(a2)
print(b2)
```

```
6
5
```

In [297…
```
0b110
```

Out[297… 6

In [298…
```
0b101
```

Out[298… 5

In [299…
```
print(0b110)
print(0b101)
```

```
6
5
```

In [300…
```
print(0b101)
print(0b110)
```

```
5
6
```

In [301…
```
#but when we use a2 + b2 then we get 11 that means we will get 4 bit which is 1 bit
print(bin(11))
print(0b1011)
```

```
0b1011
11
```



-there is other way to work using swap variable also which is XOR because it will not waste extra bit

XOR Basics

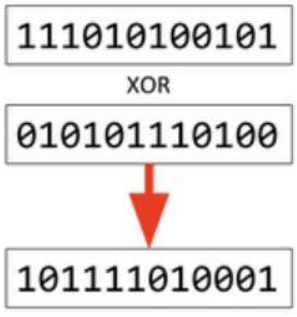
An XOR or *eXclusive OR* is a bitwise operation indicated by ^ and shown by the

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Encryption: XOR

Take data represented in binary and perform an operation against another set of bits where you get a 1 only if exactly one of the bits is 1

First Bit	Second Bit	Resulting Bit
0	0	0
0	1	1
1	0	1
1	1	0



In [302…
```
print(a2)
print(b2)
```

```
6
5
```

In [303…
```
#there is other way to work using swap variable also which is XOR because it will n
a2 = a2 ^ b2
b2 = a2 ^ b2
a2 = a2 ^ b2
```

In [304…
```
print(a2)
print(b2)
```

```
5
6
```

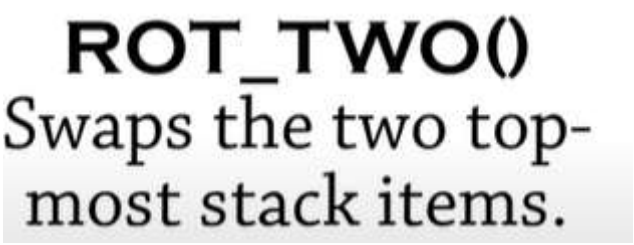
In [305…
```
a2, b2
```

Out[305… (5, 6)

In [306…
```
a2 , b2 = b2, a2 # how it work is b2 6 a2 is 5 first it goes into stack & then it r
```

In [307…
```
print(a2)
print(b2)
```

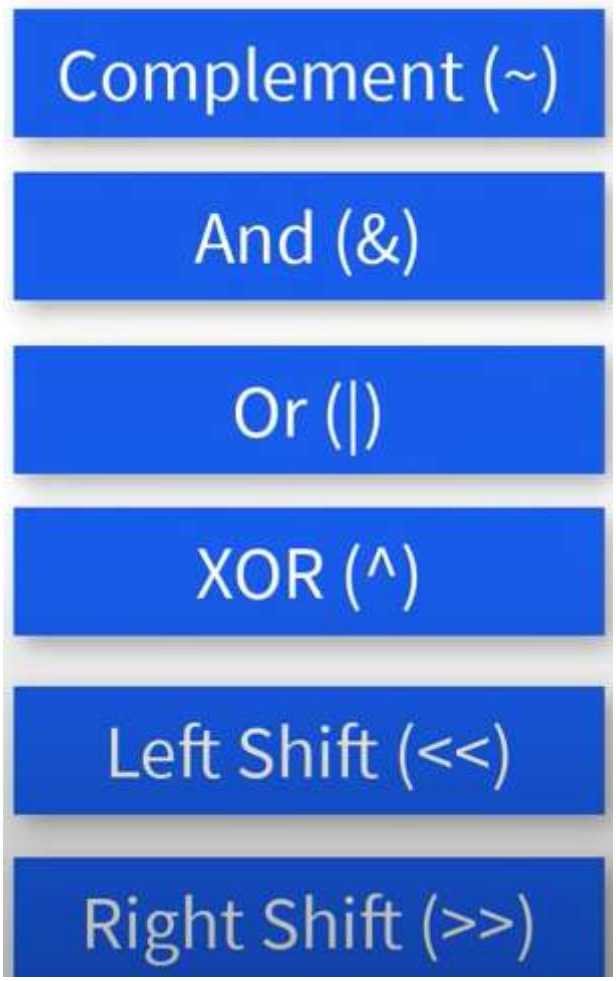
```
6
5
```



- internally it uses the rotational concept

BITWISE OPERATOR

- WE HAVE 6 OPERATORS COMPLEMENT (~) || AND (&) || OR (|) || XOR (^) || LEFT SHIFT (<<) || RIGHT SHIFT (>>)



In [308…
```
print(bin(12))
print(bin(13))
```

```
0b1100
0b1101
```

In [309…
```
0b1101
```

Out[309… 13

In [310…
```
0b1100
```

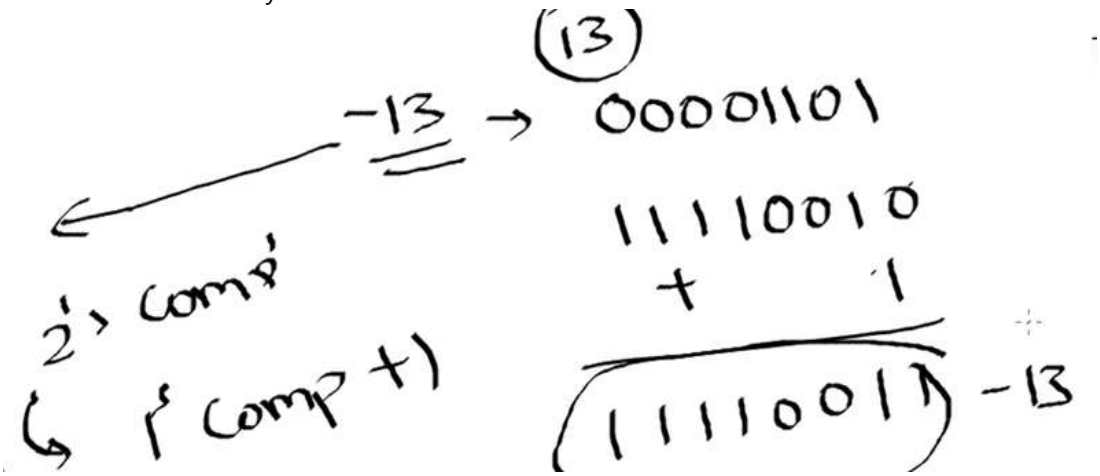
Out[310… 12

complement --> you will get this key below esc character

12 ==> 1100 ||

- first thing we need to understand what is mean by complement.

- complement means it will do reverse of the binary format i.e. - ~0 it will give you 1 ~1 it will give 0
- 12 binary format is 00001100 (complement of ~00001100 reverse the number - 11110011 which is (-13)
- in the virtual memory we cant store -ve number & the only way to store the -ve value by using complimentory
- but the question is why we got -13
- to understand this concept (we have concept of 2's complement
- 2's complement mean (1's complement + 1)
- in the system we can store +Ve number but how to store -ve number
- lets understand binary form of 13 - 00001101 + 1



In [311…
```
# COMPLEMENT (~) (TILDE  OR TILD)
~12 # why we get -13 . first we understand what is complment means (reversr of bina
```

Out[311…
```
-13
```

In [312…
```
~46
```

Out[312…
```
-47
```

In [313…
```
~54
```

Out[313…
```
-55
```

In [314…
```
~10
```

Out[314…
```
-11
```

bit wise and operator

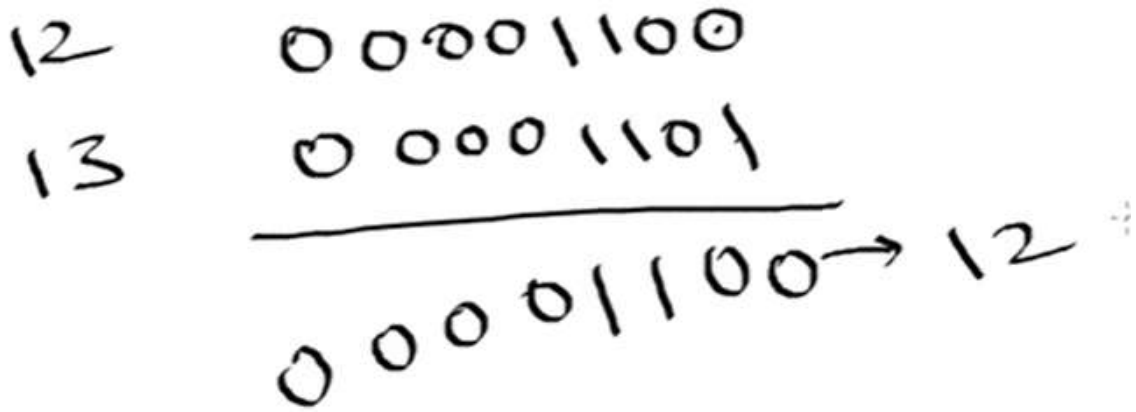
AND - LOGICAL OPERATOR ||| & - BITWISE AND OPERATOR
(we know that 1 & 1 is 1) 12 - 00001100 13 - 00001101 when we are add both then then outut we will get as 12

AND

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

OR

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

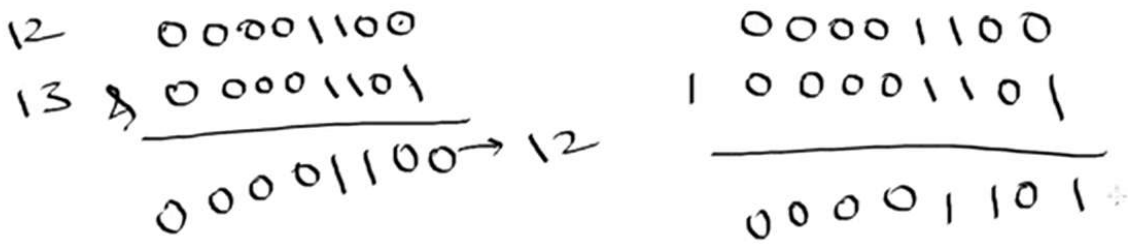


In [315…
```
12 & 13
```
Out[315… 12

In [316…
```
12 | 13
```
Out[316… 13

In [317…
```
1 & 0
```
Out[317… 0

In [318…
```
1 | 0
```
Out[318… 1



In [319…
```
bin(13)
```

Out[319…
```
'0b1101'
```

In [320…
```
print(bin(35))
print(bin(40))
```

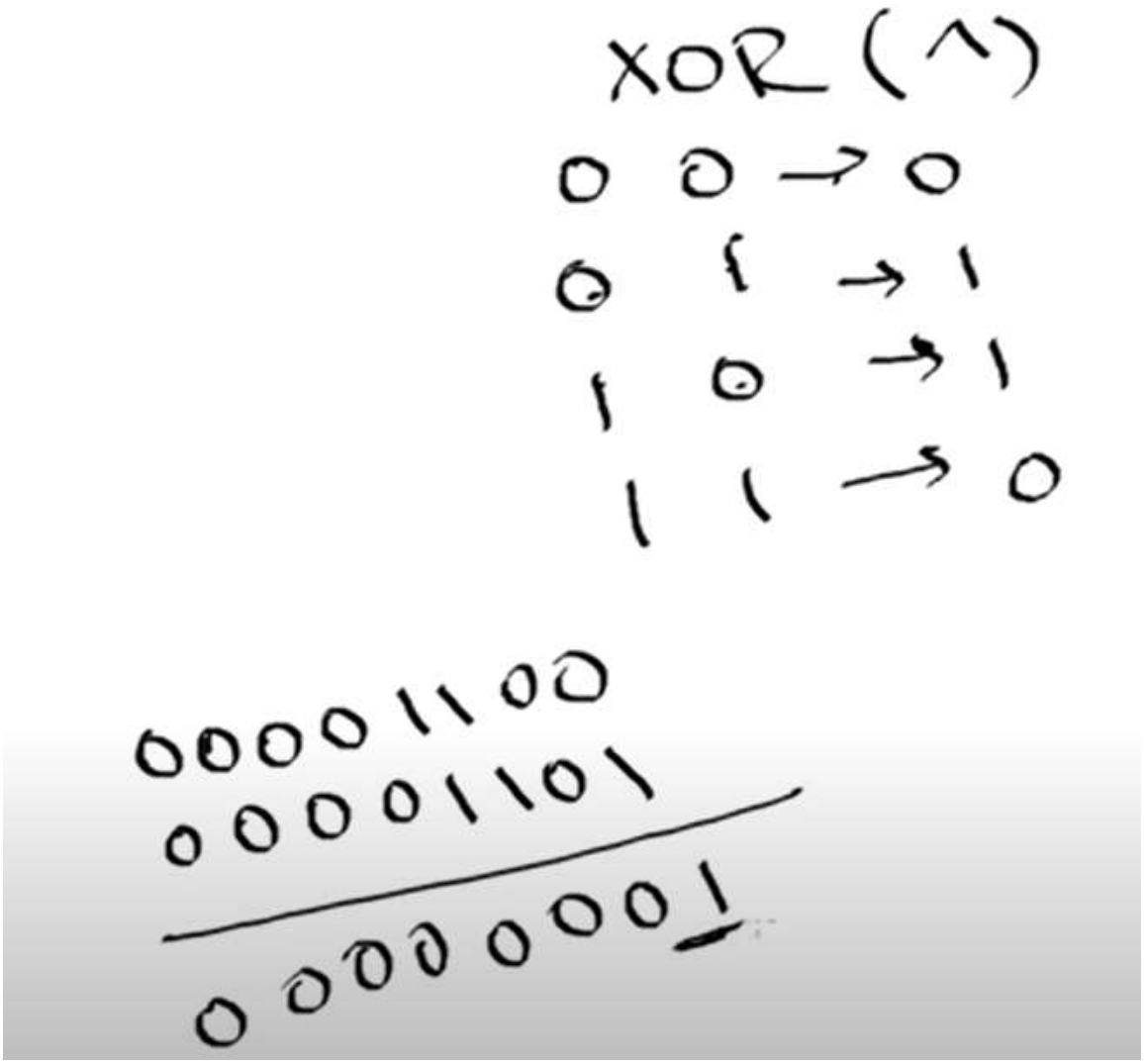
```
0b100011
0b101000
```

In [321…
```
35 & 40 #please do the homework conververt 35,40 to binary format
```

Out[321…
```
32
```

In [322…
```
35 | 40
```

Out[322…
```
43
```



In [323…
```
# in XOR if the both number are different then we will get 1 or else we will get 0
12 ^ 13
```

Out[323…
```
1
```

In [324…
```
25^30
```

Out[324…
```
7
```

In [325…
```
bin(7)
```

Out[325…
```
'0b111'
```

In [326…
```
print(bin(25))
print(bin(30))
```

```
0b11001
0b11110
```

In [327…
```
bin(25)
```

Out[327… '0b11001'

In [328…
```
bin(30)
```

Out[328… '0b11110'

In [329…
```
0b00111
```

Out[329… 7

In [330…
```
bin(10)
```

Out[330… '0b1010'

In [331…
```
10<<1
```

Out[331… 20

In [332…
```
10<<2
```

Out[332… 40



In [333…
```
bin(10)
```

Out[333… '0b1010'

In [334…
```
10<<1
```

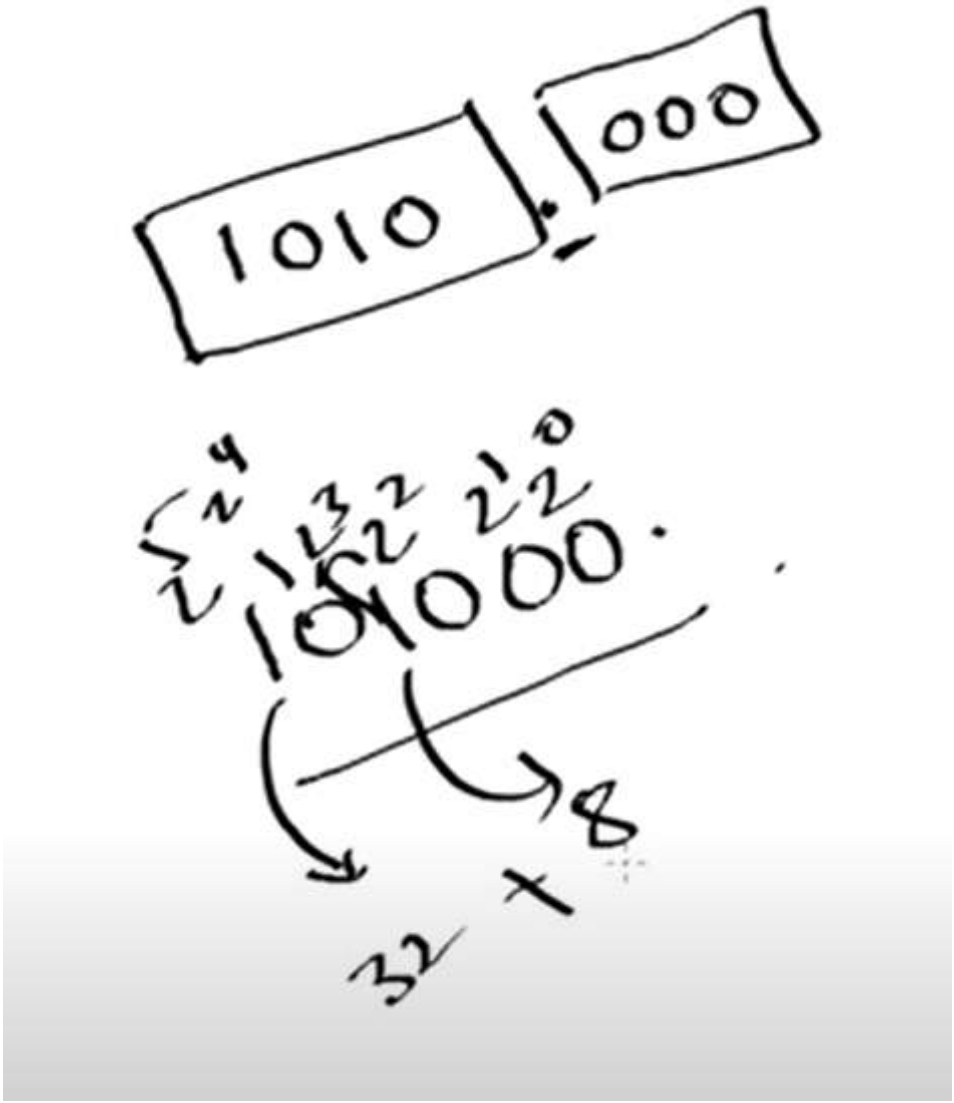
Out[334… 20

In [335…
```
# BIT WISE LEFT SHIFT OPERATOR
# in left shift what we need to to we need shift in left hand side & need to shift
#bit wise left operator bydefault you will take 2 zeros ( )
#10 binary operator is 1010 | also i can say 1010
10<<2
```

Out[335… 40

In [336…
```
10<<3
```

Out[336… 80



In [337…
```
bin(20)
```

Out[337…
```
'0b10100'
```

In [338…
```
20<<4 #can we do this
```

Out[338…
```
320
```

BITWISE RIGHTSHIFT OPERATOR

- left side we are gaining the bits
- right side we are lossing bits



In [339…
```
bin(10)
```
Out[339…
```
'0b1010'
```

In [340…
```
10 >> 1
```
Out[340…
```
5
```

In [341…
```
10>>2
```
Out[341…
```
2
```

In [342…
```
10>>3
```
Out[342…
```
1
```

In [343…
```
bin(20)
```
Out[343…
```
'0b10100'
```

In [344…
```
20>>4
```
Out[344…
```
1
```

In [345…
```
import keyword
keyword.kwlist
```

Out[345…
```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

import math function

https://docs.python.org/3/library/math.html

In [346…
```
x = sqrt(25) #sqrt is inbuild function
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[346], line 1
----> 1 x = sqrt(25) #sqrt is inbuild function

NameError: name 'sqrt' is not defined
```

In [ ]:
```
#help()
```

In [347…
```
import math # math is module
```

In [348…
```
x = math.sqrt(25)
x
```

Out[348… 5.0

In [349…
```
x1 = math.sqrt(15)
x1
```

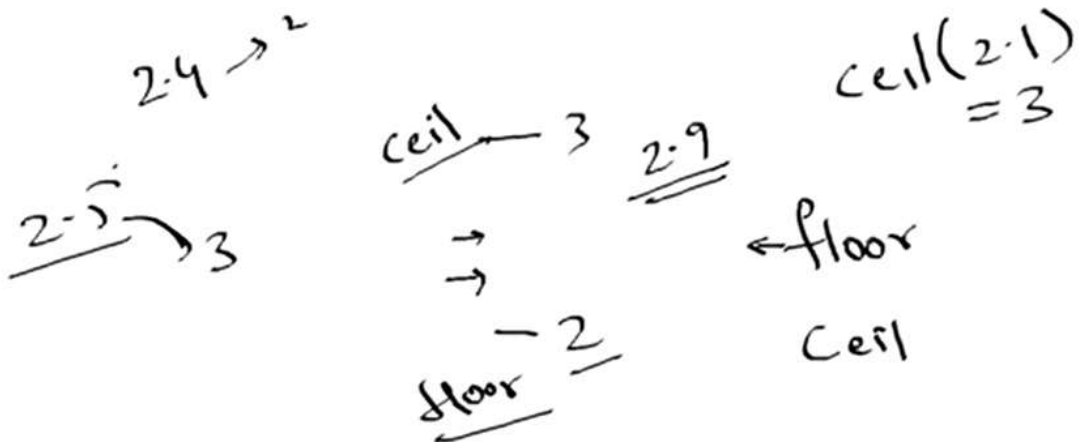
Out[349… 3.872983346207417

In [350…
```
print(math.floor(3.87)) #floor - minimum or least value
```

3

In [351…
```
print(math.ceil(3.87)) #ceil - maximum or highest value
```

4



In [352…
```
print(math.pow(3,2))
```

9.0

In [353…
```
print(math.pi) #these are constant
```

3.141592653589793

In [354…
```
print(math.e) # e - epsilon values
```

2.718281828459045

In [355…
```
m.sqrt(25)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[355], line 1
----> 1 m.sqrt(25)

AttributeError: 'int' object has no attribute 'sqrt'
```

In [2]:
```
import math as m # we need to use concept aliseing, instead of math we are using as
m.sqrt(10) #if you are lazy to type then you can use m or else you can use math
```

Out[2]: 3.1622776601683795

In [356…
```
from math import sqrt, pow # math has many function if you want to import specific
print(pow(2,3))
print(m.sqrt(10))
```

```
8.0
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[356], line 3
      1 from math import sqrt, pow # math has many function if you want to import sp
ecific function then use import
      2 print(pow(2,3))
----> 3 print(m.sqrt(10))

AttributeError: 'int' object has no attribute 'sqrt'
```

In [1]:
```
from math import sqrt, pow, floor, ceil # math has many function if you want to imp

print(pow(2,3))
print(sqrt(10))
print(floor(2.3))
print(ceil(2.3))
```

```
8.0
3.1622776601683795
2
3
```

In [3]:
```
from math import sqrt, pow,

print(pow(2,3))
print(sqrt(10))
print(floor(2.3))
print(ceil(2.3))
```

```
  Cell In[3], line 1
    from math import sqrt, pow,
                                ^
SyntaxError: trailing comma not allowed without surrounding parentheses
```

In [4]:
```
from math import *

print(pow(2,3))
print(sqrt(10))
print(floor(2.3))
print(ceil(2.3))
```

```
8.0
3.1622776601683795
2
3
```

In [5]:
```
from math import *
```

```
print(pow(2,3))
print(math.sqrt(10))
```

```
8.0
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 4
      1 from math import *
      2
      3 print(pow(2,3))
----> 4 print(math.sqrt(10))

NameError: name 'math' is not defined
```

In [6]:
```
round(pow(2,3))
```

Out[6]: 8



how to exeucute test file in using command line # pycharm run debug # how to install python idle # how to install pycharm & starts working on pycharm # run test1.py for run, debug # we are also using pycharm for loop & important deep learning & computer viso,Neural network concept

user input function in python || command line input

- how to get input from user

In [ ]:
```
x = input()
x
```

In [ ]:
```
x = input()
y = input()

z = x + y
print(z) # console is waiting for user to enter input
# also if you work in idle
```

In [ ]:
```
type(y)
```

In [ ]:
```
type(x)
```

In [ ]:
```
x1 = input('Enter the 1st number') #whenevery you works in input function it always
y1 = input('Enter the 2nd number') # it wont understand as arithmetic operator
z1 = x1 + y1
print(z1)
```

In [ ]:
```
x1 = input('user_name') #whenevery you works in input function it always give you s
y1 = input('password') # it wont understand as arithmetic operator
z1 = x1 + y1
print(z1)
```

In [ ]:
```
print(type(x1))
print(type(y1))
```

In [ ]:
```
x1 = input('Enter the 1st number') #whenevery you works in input function it always
a1 = int(x1)
y1 = input('Enter the 2nd number') # it wont understand as arithmetic operator
b1 = int(y1)
z1 = a1 + b1
print(z1)
```

for the above code notice we are using many lines because fo that wasting some memory spaces as well

In [ ]:
```
x2 = int(input('Enter the 1st number'))
y2 = int(input('Enter the 2nd number'))
z2 = x2 + y2
z2
```

lets take input from the user in char format, but we dont have char format in python

In [ ]:
```
st = input('enter a string')
print(st)
#print(type(ch))
```

In [ ]:
```
print(st[0])
```

In [ ]:
```
print(st[0:2])
```

In [ ]:
```
print(st[1])
```

In [ ]:
```
print(st[-1])
```

In [ ]:
```
st = input('enter a string')[0]
print(st)
```

In [ ]:
```
st = input('enter a string')[1]
print(st)
```

In [ ]:
```
st = input('enter a string')[5:7]
print(st)
```

In [ ]:
```
st = input('enter a string')[4:]
print(st)
```

In [ ]:
```
st = input('enter a string')
print(st) # if you enter as 2 + 6 -1 we get output as 2 + 6-1 only cuz 2+6-1 as exp
```

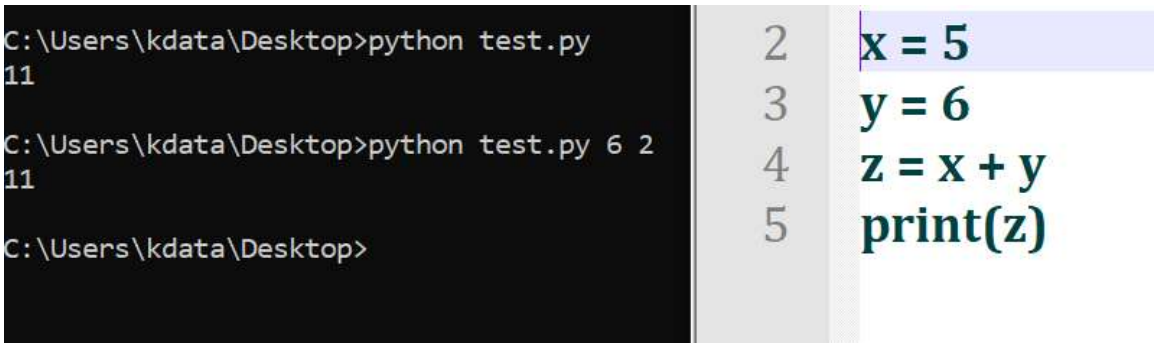
In [ ]:
```
result = input('enter an expr')
print(result)
#print(type(result))
```

In [ ]:
```
result = int(input('enter an expr'))
print(result)
```

EVAL function using input

In [ ]:
```
result = eval(input('enter an expr'))
print(result)
```

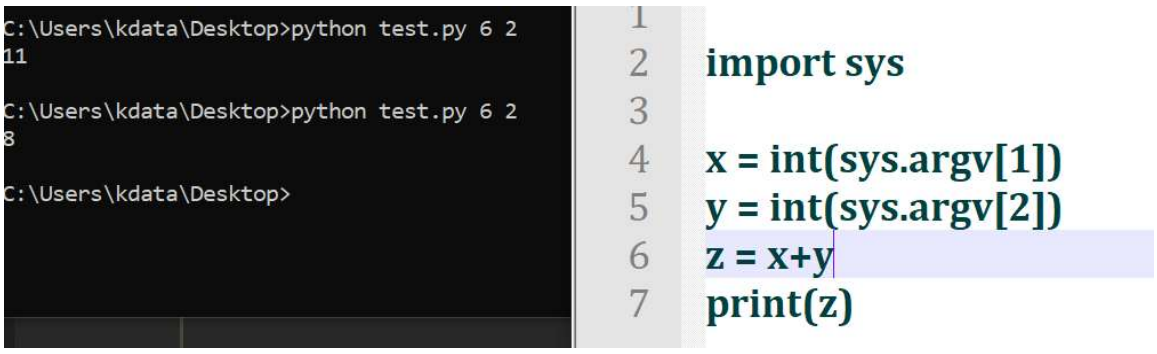
if you want to pass the value in cmd can we pass the value like this

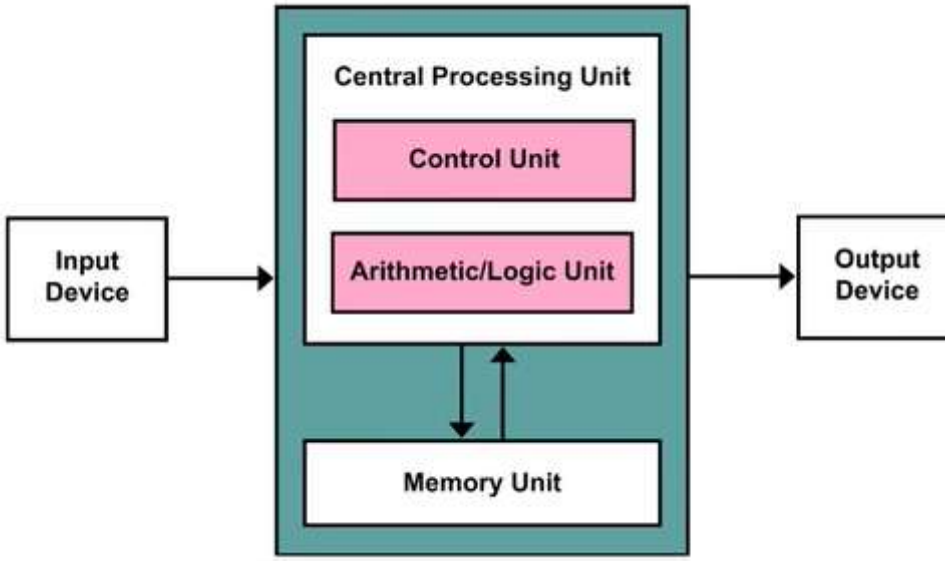


when we run the above code in cmd then we get the value to 11 only but how to add them then we need to use very important concept called (argv) -- (argument values) what it does if you pass 1 value then 1 value it will display but if you pass 2 value then it will display 2 values

argv -- it will understand based on index number & bydefault index number 0 we are use for file name , 1st index we are use for 1st argument, 2nd index we are use for 2nd argument







cpu

- cpu has 3 part - CU (Contol Unit) | MU(Memory Unit) | ALU (Arithmetic | Logic unit)
- MEMORY UNIT --> so far we work in memory unit-that means we save variable in memory & call the varible from memory & trying to save the data with the help of variable
- ALU (ARITHMETIC UNIT & LOGIC UNIT) --> ARITHMETIC UNIT --> when it comes to Arithmetic unit it performs calculation & we done so far addition, substraction, multiplication , division. LOGICAL UNIT --> when i talk about logical unit which makes your computer to think means user programed so that and let your computer think. in real life also we apply many condition to reach the goal. if this not possible then we can do that for example. e.g after complet +2 which field you can go for if not this field else what is other files. while we talk amoung each other many situation unknowily we are using if else condition.

LETS UNDERSTAND THE CONDITION TO COMPUTER TO THINK FOR THAT WE HAVE SOME SPECIAL KEYWORD -



In [7]:
```
if True: # indentiation is always 4 spaces
    print('Data Science')
```

```
Data Science
```

In [8]:
```
if True:
print('ds')
```

```
  Cell In[8], line 2
    print('ds')
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

In [9]:
```
if False:
    print('Data Science')
print('bye for now')
```

```
bye for now
```

In [10]:
```
if True: # indentiation is always 4 spaces
    print('Data Science')
print('bye for now')
```

```
Data Science
bye for now
```

In [11]:
```
if False: # indentiation is always 4 spaces
    print('Data Science')
else:
    print('bye for now')
```

```
bye for now
```

Lets do one program as if divide by 2 then reminder is 0 then it is even number if reminder is not 0 then it is odd number

In [12]:
```
#to print only even number
x = 4
r = x % 2

if r == 0:
    print('Even number')
```

```
Even number
```

In [13]:
```
#to print only even number
x = 5
r = x % 2

if r == 0:
    print('Even number')
```

In [14]:
```
x = 5
r = x % 2

if r == 0:
    print('Even number')
print('odd number')
```

```
odd number
```

In [15]:
```
x = 6
r = x % 2

if r == 0:
    print('Even number')
print('odd number')
```

```
Even number
odd number
```

In [16]:
```
x = 10
r = x % 2

if r == 0:
    print('Even number')
if r == 1:
    print('odd number')
```

```
Even number
```

In [17]:
```
x = 11
r = x % 2

if r == 0:
    print('Even number')
if r == 1:
    print('odd number')
```

```
odd number
```

In [18]:
```
x = 11
r = x % 2

if r == 0:
    print('Even number')
else:
    print('odd number')
```

```
odd number
```

In [19]:
```
x = 23
r = x % 2

if r == 0:
    print('Even number')
if r == 1:
    print('odd number')
```

```
odd number
```

In [20]:
```
x = 17
r = x % 2

if r == 0:
    print('Even number')
```

```
if r != 0:
    print('odd number')
```

```
odd number
```

if we observe the code its too many line cuz many of the coder always they wanted to reduce the code lenght which is very good practise. instead of 2 if we can use if-- else

In [21]:
```
x = 5
r = x % 2

if r == 0:
    print('Even number')
else:
    print('Odd Number')
```

```
Odd Number
```

In [22]:
```
x = 4
r = x % 2

if r == 0:
    print('Even number')
else:
    print('Odd Number')
```

```
Even number
```

NESTED IF (if we have 2 condition so we need to implment with nested if)

In [23]:
```
x = 3
r = x % 2

if r == 0:
    print('Even number')
    if x>5:
        print('greater number')

else:
    print('Odd Number')
```

```
Odd Number
```

In [24]:
```
x = 4
r = x % 2

if r == 0:
    print('Even number')
    if x>5:
        print('greater number')

else:
    print('Odd Number')
```

```
Even number
```

In [25]:
```
x = 4
r = x % 2

if r == 0:
    print('Even number')
    if x>5 :
        print('greater number')
    else:
       print('not greater ')
else:
    print('odd number')
```

```
Even number
not greater
```

In [26]:
```
x = 6
r = x % 2

if r == 0:
    print('Even number')
    if x>5 :
        print('greater number')
    else:
        print('not greater ')

else:
    print('odd number')
```

```
Even number
greater number
```

We do have concept of (IF - ELIF- ELSE) e.g i want to print (1--> one , 2 --> two, 3--> three, 4--> four, 5- five)

In [27]:
```
# when you use if it will check all condition but if we mention elif then it wont c
# when we use if condition it will check all every block of code  better debug in p
# you can debug with value 1 & d for both if & elif

x = 2

if x == 1:
    print('one')
if x == 2:
    print('Two')
if x == 3:
    print('Three')
if x == 4:
    print('four')
```

```
Two
```

In [28]:
```
# elif it wont check till the block once you find the output it wont go to next lin
# you can try with multiple parameter 1, 2 & 3 value in x

x = 4
```

```
if(x == 1):
    print('one')
elif(x == 2):
    print('Two')
elif(x == 3):
    print('Three')
elif(x == 4):
    print('four')
```

```
four
```

In [29]:
```
x = 7

if(x == 1):
    print('one')
elif(x == 2):
    print('Two')
elif(x == 3):
    print('Three')
elif(x == 4):
    print('four')
```

In [30]:
```
x = 7

if(x == 1):
    print('one')


elif(x == 2):
    print('Two')
elif(x == 3):
    print('Three')
elif(x == 4):
    print('four')

else:
    print('wrong output')
```

```
wrong output
```

In [31]:
```
x = 4

if(x == 1):
    print('one')
elif(x == 2):
    print('Two')
elif(x == 3):
    print('Three')
elif(x == 4):
    print('four')
else:
    print('wrong output')
```

```
four
```

In [32]:
```
x = 10
```

```
if(x == 1):
    print('one')
elif(x == 2):
    print('Two')
elif(x == 3):
    print('Three')
elif(x == 4):
    print('four')

else:
    print('wrong output')
```

```
wrong output
```

In [33]:
```
#short hand if
a = 30
b = 20
if a > b: print("a is greater than b")
```

```
a is greater than b
```

LOOPS -- in programing world some time we keep on repeating , may be you want to repeat 5 statement so one way is copy & paste multiple times or other way is

if you want to print the datascience 1000 times then what you will you cant copy for 1000 times , if you want to print 1000 times then you cant do manualy . that is the reason why we need to apply loop -> 2 type of loops -- While loop & For loop

In [34]:
```
print('data science')
print('data science')
print('data science')
print('data science')
print('data science')
```

```
data science
data science
data science
data science
data science
```

In [35]:
```
i = 1          # initializing

while i<=5:    # condition
    print('data science')
    i = i + 1  # increment
```

```
data science
data science
data science
data science
data science
```

In [36]:
```
i = 5          # initializing

while i>=1:    # condition
```

```
    print('data science')
    i = i - 1  # decrement
```

```
data science
data science
data science
data science
data science
```

In [37]:
```
i = 1         # initializing
while i<=5:    # condition
    print('data science : ', i)
    i = i + 1  # increment
```

```
data science :  1
data science :  2
data science :  3
data science :  4
data science :  5
```

In [38]:
```
i = 5          # initializing
while i>=1:    # condition
    print('data science :',i)
    i = i - 1  # decrement
```

```
data science : 5
data science : 4
data science : 3
data science : 2
data science : 1
```

can we use multiple while loop || nested while loop to understand nested while indepth understand you can use pycharm debug with f8 option

In [39]:
```
i = 1

while i<=5:
    print('data science') # when we mention end then new line will not create
    j = 1
    while j<=4:
        print('technology')
        j = j + 1

    i = i + 1
    print()

    # the output which we got is very lengty but how to make them one line lets ref
```

```
data science
technology
technology
technology
technology

data science
technology
technology
technology
technology

data science
technology
technology
technology
technology

data science
technology
technology
technology
technology

data science
technology
technology
technology
technology
```

In [40]:
```
i = 1
while i<=5:
    print(' datascience', end = "") # when we mention end then new line will not cr
    j = 1
    while j<=4:
        print(' technology', end="")
        j = j + 1

    i = i + 1
    print()
```

```
datascience technology technology technology technology
datascience technology technology technology technology
datascience technology technology technology technology
datascience technology technology technology technology
datascience technology technology technology technology
```

In [41]:
```
# lets use while loop usig some numbers
i = 1

while i <= 2 :
    j = 0
    while  j <= 2 :
        print(i*j, end=" ")
        j += 1
```

```
    print()
    i += 1
```

```
0 1 2
0 2 4
```

In [42]:
```
# lets use while loop usig some numbers
i = 1
while i <= 4 :
    j = 0
    while  j <= 3 :
        print(i*j, end=" ")
        j += 1
    print()
    i += 1
```

```
0 1 2 3
0 2 4 6
0 3 6 9
0 4 8 12
```

FOR LOOP - normally while loop it work with iteration or certaion some condition but for loop it will work with sequence (list, string,int)

In [43]:
```
name = 'nit'

for i in name:
    print(i)
```

```
n
i
t
```

In [44]:
```
name1 = [1,3.5,'hallo'] #i want print the value individualy

for i in name1:
    print(i)
```

```
1
3.5
hallo
```

In [45]:
```
for i in [2, 3, 7.8, 'hi']:
    print(i)
```

```
2
3
7.8
hi
```

In [46]:
```
range(5)
```

Out[46]:
```
range(0, 5)
```

In [47]:
```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

In [48]:
```
for i in range(2,5):
    print(i)
```

```
2
3
4
```

In [49]:
```
for i in range(1,10,3):
    print(i)
```

```
1
4
7
```

In [50]:
```
# print the value which is divisible by 5 table
for i in range(1,21):
    print(i)
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

In [51]:
```
# print the value which is divisible by 5

for i in range(1,51):

    if i%5==0 :
        print(i)
```

```
5
10
15
20
25
30
35
40
45
50
```

In [52]:
```python
# print the value which is divisible by 5 i dont want that value
for i in range(1,51):

    if i%5!=0 :
        print(i)
```

LETS DISCUSS ABOUT 3 KEYWORDS -- BREAK || CONTINUE || PASS BREAK STATEMNT - if you apply break statment in a loop then it will end the loop # Pass = skips block of code(function, class etc) # Continue= skips 1 step/iteration during loop # Break= jumps out of the function/loop

In [ ]:
```
# write the code user ask chocklet from vendor machne write the basic code

x = int(input('How many choclets you want:?'))

i = 1
while i<=x:
    print('choclet')
    i += 1
```

- If the user says i need 10 choclet but vending machine dont have 10 choclate & machine has only 5 choclate so what you do on those scenario

- We have 3 choice now (eiter stop the transaction by you or you can give only 5 choclate) & may be vendor machine display the result as we are out of the stock
- Now lets try in the code

In [ ]:
```
ava = 5 # the machine has only 5 choclet

x = int(input('How many choclets you want:?'))

i = 1
while i<=x:
    print('choclet')
    i += 1
# if you check the user wants 10 choclets  but availabe choclet is 5 but we got out
# in this code we just declare but we dint apply any condition to it
```

In [ ]:
```
available_choclet = 5 # the machine has only 10 candis

x = int(input('How many choclets user want:?'))

i = 1
while i<=x:

    if i>available_choclet: # we stop the execution but which code execution not en
        break # break is statement | means jump out of the loop
    print('choclet')
    i += 1

print('bye for now')
```

In [ ]:
```
available_choclet = 5 # the machine has only 10 candis

x = int(input('How many choclets you want:?'))

i = 1
while i<=x:

    if i>available_choclet: # we stop the execution but which code execution not en
        print('out of stock')
        break # break is statement | means jump out of the loop
    print('choclet')
    i += 1

print('bye for now')
```

In [1]:
```
for i in range(1,11):
    print(i)
```

```
1
2
3
4
5
6
7
8
9
10
```

- i dont want 11 number i want only 5 number for the range of 1 to 10

In [2]:
```
for i in range(1,11):
    if i == 10:
       break
    print(i)
```

```
1
2
3
4
5
6
7
8
9
```

in continue loop wont be terminate & exclue the assign number it give you entire output

In [3]:
```
for i in range(1,11):
    if i == 8:
        continue
    print(i)
```

```
1
2
3
4
5
6
7
9
10
```

In [4]:
```
for i in range(1,11):
    if i == 6:
        continue
    print('hello :',i)
```

```
hello : 1
hello : 2
hello : 3
hello : 4
hello : 5
hello : 7
hello : 8
hello : 9
hello : 10
```

#PASS Statement - pass the code & it wont go (code give you the error)

In [5]:
```
for i in range(1,11):
```

```
  Cell In[5], line 1
    for i in range(1,11):
                         ^
_IncompleteInputError: incomplete input
```

In [6]:
```
for i in range(1,11):
    pass
```

you need to print the number from 1 to 50 but dont print the number which is divisible by 3 or 5

In [7]:
```
for i in range(1,51):

    if i%3 == 0:

        print(i)
print('end')
```

```
3
6
9
12
15
18
21
24
27
30
33
36
39
42
45
48
end
```

In [8]:
```
for i in range(1,51):

    if i%3 == 0:
        continue
```

```
    print(i)
print('end')
```

```
1
2
4
5
7
8
10
11
13
14
16
17
19
20
22
23
25
26
28
29
31
32
34
35
37
38
40
41
43
44
46
47
49
50
end
```

In [9]:
```
for i in range(1,51):

    if i%3 == 0 or i%5 == 0:
        continue
    print(i)
#print('end')
# it will skip all the value which is divisible by 3 or 5
```

```
1
2
4
7
8
11
13
14
16
17
19
22
23
26
28
29
31
32
34
37
38
41
43
44
46
47
49
```

In [10]:
```python
for i in range(1,50):
    if i%3 == 0 and i%5 == 0:
        continue
    print(i)
print('end')
# when you apply and you wont get the value which is divisible by both 3 & 5 (15)
```

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
16
17
18
19
20
21
22
23
24
25
26
27
28
29
31
32
33
34
35
36
37
38
39
40
41
42
43
44
46
47
48
49
end
```

In [11]:
```python
# i dont want to print the values which are even numbers that means print only odd

for i in range(1,51):

    if (i%2 == 0):
        #print('even')
        continue
    else:
```

```
        print(i)
print('bye')
```

```
1
3
5
7
9
11
13
15
17
19
21
23
25
27
29
31
33
35
37
39
41
43
45
47
49
bye
```

PRINTING PATTERN IN PYTHON



In [12]:
```
print('# # # #')
print('# # # #')
print('# # # #')
print('# # # #')
```

```
# # # #
# # # #
# # # #
# # # #
```

In [13]:
```
for i in range(1,5):
    i=i+1
    print('# # # # ')
```

```
# # # #
# # # #
# # # #
# # # #
```

In [14]:
```
for i in range(1,5):
    if i<=5:
        print('# # # #')
```

```
# # # #
# # # #
# # # #
# # # #
```

In [15]:
```
for j in range(4):
    print('#')
```

```
#
#
#
#
```

In [16]:
```
for j in range(4):
    print('# # # #')
```

```
# # # #
# # # #
# # # #
# # # #
```

In [17]:
```
for j in range(4):
    print('#', end = " ")
```

```
# # # #
```

In [18]:
```
for j in range(4):
    print('#', end=" ")

for j in range(4):
    print('#', end=" ")
```

```
# # # # # # # #
```

In [19]:
```
for j in range(4):
    print('#', end=" ")

print()

for j in range(4):
    print('#', end=" ")
```

```
# # # #
# # # #
```

In [20]:
```
for j in range(4):
    print('#', end="  ")

print()
```

```
for j in range(4):
    print('#', end="  ")

print()

for j in range(4):
    print('#', end="  ")

print()

for j in range(4):
    print('#', end="  ")
```

```
#  #  #  #
#  #  #  #
#  #  #  #
#  #  #  #
```

In [21]:
```
for i in range(4):
    for j in range(4):
        print('#', end="  ")
    print()
    # pease use debug mode in pycharm
```

```
#  #  #  #
#  #  #  #
#  #  #  #
#  #  #  #
```



In [22]:
```
for i in range(4):
    for j in range(i+1):
        print('#', end = " ")
    print()
```

```
#
# #
# # #
# # # #
```

In [23]:
```
for i in range(1,5):
    print("# "*i)
```

```
#
# #
# # #
# # # #
```

In [24]:
```
for i in range(1,5):
    for j in range(4):
        if i>j:
            print("#",end=" ")
    print()
```

```
#
# #
# # #
# # # #
```

In [25]:
```
list(range(5))
```

Out[25]: [0, 1, 2, 3, 4]

In [26]:
```
for i in range(4):
    for j in range(i):
        print('#', end="  ")
    print()
```

```
#
#  #
#  #  #
```

In [27]:
```
for i in range(4):
    for j in range(i+1):
        print('#', end="  ")
    print()
```

```
#
#  #
#  #  #
#  #  #  #
```

```
# # # #
# # #
# #
#
```

In [28]:
```
for i in range(4):
    for j in range(4-i):
        print('#', end="  ")
    print()
```

```
#  #  #  #
#  #  #
#  #
#
```

In [29]:
```
for i in range(1,5):
    print("# "*(5-i))
```

```
# # # #
# # #
# #
#
```



- For|Else in python
- In other language for else not supportable but in python it is supportable

eg- lets print the number from 1- 20 & we dont want print number which is divisible by 5

In [30]:
```
nums = [12,15,18,21,26]

for num in nums:
    if num % 5 == 0:
        print(num)
```

```
15
```

In [31]:
```
nums = [12,14,18,21,25,30,35]

for num in nums:
    if num % 5 == 0:
        print(num)
```

```
25
30
35
```

In [32]:
```
nums = [12,14,18,21,25,20]

for num in nums:
    if num % 5 == 0:
        print(num)
```

```
25
20
```

In [33]:
```
nums = [12,14,18,21,20,25]

for num in nums:
    if num % 5 == 0:
        print(num)
        break
```

```
20
```

In [34]:
```
nums = [10,14,18,21,5,10]

for num in nums:
    if num % 5 == 0:
```

```
        print(num)
        break #it will print only 1 number then it break
```

```
10
```

In [35]:
```
nums = [7,14,18,21,23,27] #hear there is no number which is divisible by 5 we got o

for num in nums:
    if num % 5 == 0:
        print(num)
        break
else:
    print('number not found')
```

```
number not found
```

In [36]:
```
nums = [7,14,18,21,23,27,29] #hear there is no number which is divisible by 5 we go

for num in nums:
    if num % 5 == 0:
        print(num)
        break
    else:
        print('Number Not Found') #every iteration it cheking condition
```

```
Number Not Found
Number Not Found
Number Not Found
Number Not Found
Number Not Found
Number Not Found
Number Not Found
```

In [37]:
```
nums = [7,14] #hear there is no number which is divisible by 5 we got output as bla
for num in nums:
    if num % 5 == 0:
        print(num)
        break
    else:
        print('Number Not Found') #every iteration it cheking condition
```

```
Number Not Found
Number Not Found
```

In [38]:
```
nums = [7,14,18,21,23,27] #hear there is no number which is divisible by 5 we got o

for num in nums:
    if num % 5 == 0:
        print(num)
        break
else:
        print('Number Not Found') # hear else we dont write in if block but we can
```

```
Number Not Found
```

In [39]:
```
nums = [10,14,18,21,20,27] #hear there is no number which is divisible by 5 we got

for num in nums:
```

```
    if num % 5 == 0:
        print(num)
        break
else:
        print('Not Found')
```

```
10
```

In [40]:
```
nums = [10,14,18,21,20,27,30] #hear there is no number which is divisible by 5 we g

for num in nums:
    if num % 5 == 0:
        print(num)
        #break
else:
        print('Not Found')
```

```
10
20
30
Not Found
```

In [41]:
```
nums = [10,14,18,21,20,27] #hear there is no number which is divisible by 5 we got
for num in nums:
    if num % 5 == 0:
        print(num)
        break
else:
        print('Not Found')
```

```
10
```

- prime number - how to check given number is prime number or not



In [42]:
```
num = 13

for i in range(2,num):
    if num % i == 0:
        print('Not prime Number')
        break
else:
    print('Prime Number')
```

```
Prime Number
```

In [43]:
```
num = 13
for i in range(2,num):
    if num % i == 0:
        print('Not prime Number')
```

```
        break
else:
    print('Prime Number')
```

```
Prime Number
```



In [ ]:
```
from array import *
arr = array('i',[])

n = int(input('Enter the length of the array'))

for i in range(5):
    x = int(input('Enter the next value'))
    arr.append(x)
print(arr)
```

In [ ]:
```
from array import *
arr = array('i',[])

n = int(input('Enter the length of the array'))

for i in range(5):
    x = int(input('Enter the next value'))
    arr.append(x)
print(arr)
```

In [ ]:
```
from array import *
arr = array('i',[])

n = input('Enter the length of the array')

for i in range(5):
    x = input('Enter the next value')
    arr.append(x)
print(arr)
```

Way of creating array using numpy

In [1]:
```
from numpy import *
arr = array([1,2,3,4,5])
print(arr)
type(arr)
```

```
[1 2 3 4 5]
```

Out[1]:
```
numpy.ndarray
```

In [2]:
```
print(arr.dtype)
```

```
int64
```

In [3]:
```
arr = array([1,2,3,4,5.9])
print(arr)
```

```
[1.  2.  3.  4.  5.9]
```

In [4]:
```
print(arr.dtype)
```

```
float64
```

In [5]:
```
arr2 = array([1,2,3,4,5.9],float)
arr2
```

Out[5]:
```
array([1. , 2. , 3. , 4. , 5.9])
```

In [6]:
```
arr3 = array([1,2,3,4,5.6],int)
arr3
```

Out[6]:
```
array([1, 2, 3, 4, 5])
```

In [7]:
```
import numpy as np
```

In [8]:
```
arr4 = np.linspace(0, 16, 10) # break the code between 10 spaces between 0 to 16 bu
arr4
```

Out[8]:
```
array([ 0.        ,  1.77777778,  3.55555556,  5.33333333,  7.11111111,
        8.88888889, 10.66666667, 12.44444444, 14.22222222, 16.        ])
```

In [9]:
```
arr5 = np.arange(0,10,2) # arange - as range
arr5
```

Out[9]:
```
array([0, 2, 4, 6, 8])
```

In [10]:
```
arr6 = np.zeros(5)
arr6
```

Out[10]:
```
array([0., 0., 0., 0., 0.])
```

In [11]:
```
arr7 = np.ones(5)
arr7
```

Out[11]:
```
array([1., 1., 1., 1., 1.])
```

In [ ]:
```
import tkinter as tk
# Function to be called when the button is clicked
def on_button_click():
    label.config(text="Button clicked!")

# Create the main application window
root = tk.Tk()
root.title("Simple Tkinter App")
```

```
# Create a label widget
label = tk.Label(root, text="Hello, Tkinter!")
label.pack(pady=20)

# Create a button widget
button = tk.Button(root, text="Click Me", command=on_button_click)
button.pack(pady=20)

# Run the application
root.mainloop()
```

In [ ]:
```
import tkinter as tk
from tkinter import messagebox

# Function to be called when the button is clicked
def on_button_click():
    user_input = entry.get()
    messagebox.showinfo("Information", f"You entered: {user_input}")

# Create the main application window
root = tk.Tk()
root.title("Simple Tkinter App")

# Create a label widget
label = tk.Label(root, text="Enter something:")
label.pack(pady=10)

# Create a text entry widget
entry = tk.Entry(root, width=30)
entry.pack(pady=10)

# Create a button widget
button = tk.Button(root, text="Submit", command=on_button_click)
button.pack(pady=10)

# Run the application
root.mainloop()
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: